

Chronico Diabo - Architecture and Requirements Documentation

Generated: 2026-05-16

This document maps the cahier des charges for the M1 Big Data UGC and Digital Mobile project to the current Chronico Diabo codebase. The source PDF is unnumbered, so the 48 requirements below are the extracted, numbered traceability version of its objectives, expected work, deliverables, evaluation criteria, constraints, bonus items, and expected outcome.

Important filename note: the project currently implements some requested concepts under different file names. There is no `src/lib/nlp/`, no `src/lib/emotion-detection.ts`, and no `src/lib/companion-memory.ts`. Their current equivalents are `notebooks/pipeline_nlp.ipynb`, `src/lib/emotion.ts`, `src/lib/embeddings.ts`, `src/lib/db/kb.ts`, `src/lib/db/companion.ts`, and the memory-building code inside `src/app/api/chat/route.ts`.

Architecture Overview

Diabo is a Next.js 16 App Router application for people living with diabetes. It combines an empathetic LLM companion, RAG over a curated diabetes knowledge base, Hugging Face NLP models, a Rive avatar, health tracking screens, place recommendation modules, authentication, reminders, Web Push, and localized French/Arabic UI.

Core technologies:

Area	Technologies
Web app	Next.js 16, React 19, TypeScript, Tailwind CSS
LLM	Vercel AI SDK, OpenRouter free models, structured tool calling
NLP	Hugging Face Inference API, cardiffnlp/twitter-xlm-roberta-base-sentiment, sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
RAG and storage	MongoDB Atlas, Atlas Vector Search, MongoDB Node driver
Avatar	@rive-app/react-canvas, public/diabo.riv, DiaboCon ViewModel
Auth	Auth.js / NextAuth v5, Google OAuth, Credentials provider, bcrypt
i18n and theme	next-intl, locale routing, RTL layout, next-themes
Health UI	Recharts, manual glucose logging, LLM weekly summary
Maps and places	Google Places API optional source, Overpass fallback, MapLibre GL, OpenFreeMap tiles
Notifications	Web Push, VAPID, service worker registration

Part 1 - Requirements Mapping

Requirement Traceability Matrix

#	Requirement text	Primary files	How the files satisfy it	Technologies and key sections
1	Conceive an intelligent AI companion, not a simple chatbot.	src/app/api/chat/route.ts, src/lib/diabo/persona.ts, src/components/chat/ChatPanel.tsx, src/components/diabo/DiaboProvider.tsx	The chat endpoint builds a persona-driven system prompt, injects user context, RAG context, page context, and health safety overrides. The UI binds the conversation to avatar state.	AI SDK streamText, DIABO_PERSONA_FR, buildMemoryBlock, buildPageContextBlock, useChat.
2	Understand natural user messages.	src/app/api/chat/route.ts, src/lib/llm.ts	User messages are converted to model messages and streamed to an LLM capable of natural language understanding in French, Arabic, and English.	convertToModelMessages, getChatModel, OpenRouter chat model.
3	Provide advice to the user.	src/lib/diabo/persona.ts, src/app/api/chat/route.ts, src/lib/diabo/knowledge.ts	The persona defines the advising role and limits. Retrieved diabetes knowledge chunks provide practical context on food, activity, monitoring, hypo/hyperglycemia, travel, Ramadan, and pregnancy.	Prompt engineering, RAG, curated KB seed chunks.

#	Requirement text	Primary files	How the files satisfy it	Technologies and key sections
4	Accompany the user over time and consider context.	src/lib/db/chats.ts, src/lib/db/companion.ts, src/app/api/chat/route.ts, src/components/chat/ChatPanel.tsx	Conversations persist in MongoDB; signed-in users can keep multiple conversations; companion profile fields are injected into the LLM prompt.	MongoDB collections chats, messages, companion_profiles; getProfile, appendMessage.
5	Interact naturally and empathetically.	src/lib/diabo/persona.ts, src/lib/emotion.ts, src/lib/diabo/emotion-map.ts, src/components/diabo/DiaboProvider.tsx	The prompt forces validation before advice. Sentiment results drive Diabo's face to concerned, neutral, or happy presets.	Hugging Face sentiment, emotionToPreset, applyEmotion, Rive ViewModel.
6	Use real textual data.	notebooks/google_play_reviews.csv, notebooks/app_store_reviews.csv, notebooks/posts_reddit.csv, notebooks/commentaires_reddit.csv, notebooks/hackernews.csv	The offline NLP deliverable contains real reviews, posts, comments, and HN documents used for market and sentiment analysis.	CSV corpora, public review/platform data, notebook pipeline.
7	Apply advanced NLP techniques.	notebooks/pipeline_nlp.ipynb, notebooks/README.md, src/lib/emotion.ts, src/lib/embeddings.ts	The notebook covers cleaning, representation, sentiment, topic modeling, zero-shot classification, and visualization. The app reuses the same sentiment and embedding families live.	Hugging Face, BERTopic, zero-shot classification, multilingual MiniLM embeddings.
8	Integrate LLM models.	src/lib/llm.ts, src/app/api/chat/route.ts, src/app/api/glucose/summary/route.ts, src/lib/restaurant-scorer.ts, src/lib/hotel-scorer.ts	OpenRouter is wrapped once and reused for chat, summaries, and structured scoring. Fallback models can be configured through environment variables.	createOpenRouter, getChatModel, AI SDK streamText, generateText, generateObject.
9	Include emotion and empathy as mandatory capabilities.	src/app/api/emotion/route.ts, src/lib/emotion.ts, src/lib/diabo/emotion-map.ts, src/components/chat/ChatPanel.tsx	Each submitted user message is sent to /api/emotion in parallel with chat. The returned label updates Diabo's facial preset.	HF text classification, analyzeEmotion, client-side applyEmotion.
10	Build a useful and credible companion.	src/lib/diabo/persona.ts, src/lib/diabo/knowledge.ts, src/app/api/chat/route.ts, src/components/glucose/GlucoseTracker.tsx	Health advice is constrained by safety language, curated KB context, and clear escalation rules. Glucose features are manual and transparent rather than pretending to integrate medical devices.	Safety prompt, RAG relevance filter, glucose dashboard, medical disclaimers.
11	Collect data from articles and comments.	notebooks/README.md, notebooks/*.csv, docs/project-pitch.pdf	The notebook directory documents review/comment sources. The project pitch captures the marketing/benchmark analysis derived from those datasets.	Google Play reviews, App Store RSS, Reddit/PullPush, Hacker News Algolia.
12	Justify data sources.	notebooks/README.md, README.md, docs/project-pitch.pdf	The README and notebook README explain why app reviews and social discussions are relevant to companion app expectations and empathy gaps.	Dataset summary tables and benchmark synthesis.
13	Document data collection method.	notebooks/README.md, notebooks/pipeline_nlp.ipynb	Reproduction steps describe uploading CSVs, running the Colab-ready notebook, and generating enriched analysis outputs.	Jupyter/Colab workflow, CSV inputs, reproducible pipeline.
14	Include a digital marketing strategy for domain understanding.	docs/project-pitch.pdf, README.md, notebooks/benchmark_synthese.csv	Market positioning is documented in the pitch and summarized in the README: empathy is a high-value gap in diabetes apps.	Benchmark synthesis and positioning narrative.
15	Diagnose the current market.	notebooks/pipeline_nlp.ipynb, notebooks/benchmark_synthese.csv, docs/project-pitch.pdf	The notebook and output CSV quantify sentiment and dominant frustrations across competing apps.	Sentiment scoring, topic modeling, review aggregation.
16	Benchmark AI companions in general.	notebooks/posts_reddit.csv, notebooks/commentaires_reddit.csv, notebooks/hackernews.csv, docs/project-pitch.pdf	Reddit and HN datasets cover companion/chatbot discourse beyond diabetes apps.	Social/web text mining, zero-shot theme labels.
17	Benchmark companions in the selected domain.	notebooks/google_play_reviews.csv, notebooks/app_store_reviews.csv, notebooks/benchmark_synthese.csv	Health and diabetes app reviews are analyzed for empathy, UX, usefulness, paywall frustration, and missing features.	App review NLP benchmark.
18	Segment the market.	docs/project-pitch.pdf, README.md, src/components/onboarding/OnboardingFlow.tsx	Market segmentation is presented in the pitch; app onboarding operationalizes segmentation by diabetes type and user goal.	Persona-driven intake fields.
19	Define at least two personas.	docs/project-pitch.pdf, src/components/onboarding/OnboardingFlow.tsx, src/app/api/chat/route.ts	Personas are documented in the pitch; the app captures profile/persona-like preferences and injects them into the LLM prompt.	buildOnboardingProfileBlock, local/profile storage.

#	Requirement text	Primary files	How the files satisfy it	Technologies and key sections
20	Clean textual data.	notebooks/pipeline_nlp.ipynb, notebooks/README.md	The notebook's first module handles cleanup before representation and analysis.	Pandas/text preprocessing pipeline.
21	Preprocess textual data.	notebooks/pipeline_nlp.ipynb, src/lib/emotion.ts,src/lib/embeddings.ts	Offline preprocessing prepares corpora; live code trims and length-limits user text for model calls.	<code>text.trim(),.slice(0, 1000),.slice(0, 2000)</code> .
22	Represent text numerically.	notebooks/pipeline_nlp.ipynb, src/lib/embeddings.ts, src/lib/db/kb.ts	Offline and live code use multilingual sentence embeddings. KB chunks and user queries share the same vector space.	MiniLM, EMBEDDING_DIM = 384, embedText, Atlas vector index.
23	Perform sentiment analysis.	notebooks/pipeline_nlp.ipynb, src/lib/emotion.ts,src/app/api/emotion/route.ts	The validated Cardiff multilingual sentiment model is used in both the notebook and the live API.	cardiffnlp/twitter-xlm-roberta-base-sentiment, textClassification.
24	Perform classification.	notebooks/pipeline_nlp.ipynb, src/app/api/chat/route.ts	Offline zero-shot classification categorizes benchmark themes. Live chat classifies emergency intent through keyword detection and routes restaurant/hotel intents through LLM tools.	Zero-shot notebook, detectEmergencyIntent, tool calling.
25	Provide summarization.	notebooks/pipeline_nlp.ipynb, src/app/api/glucose/summary/route.ts, src/app/api/chat/translate/route.ts	The app generates concise weekly glucose summaries from logged measurements. Chat translation also reconstructs conversation content for Arabic output.	AI SDK generateText, concise prompt, max output tokens.
26	Extract information / NER-like entities.	notebooks/pipeline_nlp.ipynb, src/lib/places/google-places.ts, src/lib/overpass.ts, src/lib/overpass-hotels.ts	The offline pipeline includes information extraction tasks. In the app, structured location/place data are extracted and normalized from Places/OSM APIs.	normalizeRestaurants, normalizeHotels, cuisine/address/phone extraction.
27	Analyze and classify pipeline outputs.	notebooks/pipeline_nlp.ipynb, notebooks/benchmark_synthese.csv, README.md	Benchmark synthesis and README headline findings translate NLP outputs into product direction.	Sentiment correlation, dominant theme labels.
28	Integrate a language model for interaction.	src/app/api/chat/route.ts, src/lib/llm.ts,src/components/chat/ChatPanel.tsx	User messages stream through a server route to the OpenRouter model and back to the client with AI SDK UI messages.	AI SDK transport, streamText, SSE-style UI message stream.
29	Use prompt engineering.	src/lib/diabo/persona.ts, src/app/api/chat/route.ts, src/app/api/glucose/summary/route.ts, src/lib/restaurant-scorer.ts	The project uses layered prompts for persona, KB context, page context, profile memory, safety overrides, and structured scoring.	DIABO_PERSONA_FR, augmentedSystem, structured JSON schemas.
30	Implement RAG where recommended.	src/lib/db/kb.ts,src/lib/embeddings.ts, src/lib/diabo/knowledge.ts, src/app/api/admin/seed-kb/route.ts, src/app/api/chat/route.ts	KB chunks are embedded, stored in MongoDB Atlas Vector Search, retrieved by query, filtered by score, and inserted into the system prompt.	\$vectorSearch, searchKb, upsertChunk, KB_SEED.
31	Generate contextualized responses.	src/app/api/chat/route.ts, src/lib/db/companion.ts, src/lib/db/kb.ts,src/components/chat/ChatPanel.tsx	Responses are contextualized by retrieved KB chunks, user profile, onboarding data, page context, selected chat, and emergency intent.	buildMemoryBlock, buildOnboardingProfileBlock, buildPageContextBlock.
32	Detect emotions.	src/lib/emotion.ts,src/app/api/emotion/route.ts	The live API classifies user text into positive, neutral, or negative labels with confidence scores.	analyzeEmotion, normalizeLabel, HF Inference API.
33	Adapt responses to emotional state.	src/lib/diabo/persona.ts, src/lib/diabo/emotion-map.ts, src/components/diabo/DiaboProvider.tsx, src/components/chat/ChatPanel.tsx	The persona requires empathy-first replies; the avatar visually adapts to the detected sentiment.	emotionToPreset, applyEmotion, empathetic system prompt.
34	Act as an intelligent adviser.	src/lib/diabo/persona.ts, src/app/api/chat/route.ts, src/lib/restaurants/tool.ts, src/lib/hotels/tool.ts	Diabo advises on diabetes daily life and can call tools for restaurant and hotel recommendations.	AI SDK tools findRestaurants, findHotels.
35	Provide an application interface.	src/app/[locale]/layout.tsx, src/app/[locale]/page.tsx, src/components/nav/Sidebar.tsx, src/app/globals.css	The localized app shell, sidebar, responsive layout, and page components provide the product interface.	Next.js App Router, server components, client components.

#	Requirement text	Primary files	How the files satisfy it	Technologies and key sections
36	Provide a chatbot interface.	src/components/chat/ChatPanel.tsx, src/app/api/chat/route.ts, src/app/api/chats/current/messages/route.ts, src/app/api/chats/[id]/messages/route.ts	The chat UI supports submit, stop, regenerate, reset, hydration, history, and persisted metadata.	useChat, DefaultChatTransport, Mongo history APIs.
37	Include an avatar.	public/diabo.riv, src/components/diabo/DiaboStage.tsx, src/components/diabo/DiaboProvider.tsx, src/lib/diabo/types.ts	Rive renders Diabo and receives state through typed ViewModel properties only.	@rive-app/react-canvas, useViewModelInstanceBoolean, useViewModelInstanceNumber.
38	Integrate NLP modules in the app.	src/app/api/emotion/route.ts, src/lib/emotion.ts, src/lib/embeddings.ts, src/lib/db/kb.ts	Sentiment analysis and embeddings are live server modules used by chat/emotion/RAG flows.	HF SDK, text classification, feature extraction, vector search.
39	Integrate LLM modules in the app.	src/lib/llm.ts, src/app/api/chat/route.ts, src/app/api/glucose/summary/route.ts, src/lib/restaurant-scorer.ts, src/lib/hotel-scorer.ts	A central LLM provider supports chat, summaries, and structured recommendation scoring.	OpenRouter, AI SDK.
40	Integrate an emotion module in the app.	src/app/api/emotion/route.ts, src/components/chat/ChatPanel.tsx, src/components/diabo/DiaboProvider.tsx, src/lib/diabo/emotion-map.ts	The emotion endpoint is called for each user turn and the result controls avatar state.	Fire-and-forget emotion fetch, Rive preset patching.
41	Store application data.	src/lib/mongodb.ts, src/lib/db/*.ts, src/app/api/chats/*.ts, src/app/api/glucose/*.ts, src/app/api/reminders/*.ts	MongoDB stores chats, messages, KB chunks, glucose logs, users, reminders, recommendation cache, and companion profiles.	MongoDB driver, indexes, collection helpers.
42	Deliver a functional app integrating NLP, LLM, and empathy.	README.md, src/app/[locale]/page.tsx, src/app/api/chat/route.ts, src/app/api/emotion/route.ts, src/components/diabo/*	README sprint status shows implemented demos. The home page combines chat, LLM, emotion, and avatar.	Next.js app shell, AI SDK, Rive, HF.
43	Produce a report covering introduction, data, NLP, emotion, LLM, architecture, implementation, results, and discussion.	docs/architecture-documentation.md, notebooks/README.md, README.md, progress.txt	This documentation and existing project notes provide the architecture/implementation base for the final report.	Markdown documentation, traceability matrix.
44	Prepare a 10-15 minute presentation.	docs/project-pitch.pdf, README.md, docs/architecture-documentation.md	The pitch deck is the presentation artifact; this document supplies technical mapping for architecture slides.	PDF pitch deck, architecture documentation.
45	Meet LLM quality and relevance evaluation criteria.	src/lib/diabo/persona.ts, src/lib/llm.ts, src/app/api/chat/route.ts, src/lib/diabo/knowledge.ts	Prompt constraints, fallback models, RAG context, and safety rules improve response quality and relevance.	Model fallback list, retrieval score filter, system prompt.
46	Meet NLP pipeline and analysis evaluation criteria.	notebooks/pipeline_nlp.ipynb, notebooks/benchmark_synthese.csv, src/lib/emotion.ts, src/lib/embeddings.ts	Offline analysis demonstrates the NLP pipeline; live modules reuse validated models.	Sentiment validation, embeddings, benchmark synthesis.
47	Meet empathy, originality, feature richness, real-world value, and architecture criteria.	src/components/diabo/*, src/components/glucose/*, src/components/restaurants/*, src/components/hotels/*, src/components/reminders/*, src/components/settings/*	The application combines avatar empathy, diabetes-specific health tracking, place recommendations, reminders, multilingual UI, and structured architecture.	Rive, Recharts, MapLibre, Web Push, next-intl.
48	Respect constraints and bonus expectations: group work, deadlines, commented code, no plagiarism, multilingual, deployment, advanced UX.	README.md, AGENTS.md, src/lib/diabo/knowledge.ts, src/i18n/*, messages/*, src/app/[locale]/layout.tsx, src/components/nav/*	Sprint status tracks progress. Comments document complex modules. KB states no copy-paste. FR/AR, RTL, dark mode, responsive nav, command palette, and deploy-ready structure address bonus UX/deployment expectations.	next-intl, RTL dir, next-themes, responsive sidebar, code comments.

Key Implementation Areas

Companion IA Intelligent - LLM Integration

Primary files:

File	Role
src/lib/llm.ts	Lazy OpenRouter provider setup and model fallback chain through getChatModel().
src/app/api/chat/route.ts	Main chat orchestration: auth/session resolution, persistence, RAG, profile memory, safety override, tool calling, streaming, metadata persistence.
src/lib/diabo/persona.ts	French/Maghreb-focused Diabo system prompt with empathy, diabetes advice boundaries, multilingual behavior, and emergency escalation.
src/lib/db/companion.ts	Current companion memory/profile store. This is the practical equivalent of the requested src/lib/companion-memory.ts.
src/components/chat/ChatPanel.tsx	Client chat state, transport, history hydration, avatar lifecycle, and emotion request wiring.

Key code sections:

Function / section	Purpose
getOpenRouter()	Creates the OpenRouter provider only when needed and validates API key availability.
getChatModel()	Returns the configured primary chat model with optional OpenRouter native fallback models.
POST() in /api/chat	Main server workflow for a user turn.
detectEmergencyIntent()	Detects urgent medical phrases and injects SAFETY_OVERRIDE.
buildMemoryBlock()	Converts companion profile fields into a compact prompt context.
buildOnboardingProfileBlock()	Adds onboarding preferences to the prompt.
streamText({... tools ...})	Streams the answer and exposes findRestaurants and findHotels tools.

NLP Pipeline

Primary files:

File	Role
notebooks/pipeline_nlp.ipynb	Offline reproducible NLP deliverable: cleanup, representation, sentiment, topic modeling, zero-shot classification, visualization, benchmark synthesis.
notebooks/README.md	Documents corpora, models, headline finding, and reproduction steps.
src/lib/emotion.ts	Live sentiment analysis on user messages with the same Cardiff multilingual model used offline.
src/lib/embeddings.ts	Live multilingual sentence embedding helper for RAG.
src/lib/db/kb.ts	Vector search over embedded diabetes KB chunks.

Current structure note: there is no src/lib/nlp/ directory. The live NLP surface is split by responsibility into emotion.ts, embeddings.ts, and db/kb.ts, while the full academic NLP pipeline remains in notebooks/.

Emotion Module

Primary files:

File	Role
src/lib/emotion.ts	Server-only Hugging Face sentiment classifier. Equivalent to the requested emotion-detection.ts.
src/app/api/emotion/route.ts	HTTP route for { text } -> { label, score, scores }.
src/lib/diabo/emotion-map.ts	Maps positive/neutral/negative labels to Diabo presets.

File	Role
src/components/chat/ChatPanel.tsx	Calls /api/emotion after submit and applies avatar emotion.
src/components/diabo/DiaboProvider.tsx	Holds avatar state and exposes applyEmotion().

Key behavior:

Detected label	Avatar preset	Intent
positive	happy	Celebrate with the user.
neutral	neutral	Remain attentive.
negative	worried	Show concern without amplifying sadness.

Chatbot Interface

Primary files:

File	Role
src/components/chat/ChatPanel.tsx	Main chat shell, message renderer, input bar, empty state, loading state, safety banner, hydration, reset, regenerate, stop.
src/components/chat/ConversationSidebar.tsx	Signed-in conversation list, selection, and Arabic translation trigger.
src/app/api/chats/route.ts	Lists user conversations.
src/app/api/chats/current/messages/route.ts	Hydrates anonymous/current chat.
src/app/api/chats/[id]/messages/route.ts	Hydrates selected signed-in chat.
src/app/api/chats/reset/route.ts	Resets anonymous cookie chat.
src/app/api/chats/transfer-anon/route.ts	Moves an anonymous chat to a signed-in user.

Technologies: AI SDK React useChat, DefaultChatTransport, Next.js route handlers, MongoDB.

Avatar

Primary files:

File	Role
public/diabo.riv	Runtime Rive export of the Diabo avatar.
src/lib/diabo/types.ts	Typed contract for DiaboCon; defines state machine names, numeric channels, booleans, and presets.
src/components/diabo/DiaboProvider.tsx	React state authority for avatar lifecycle and emotion.
src/components/diabo/DiaboStage.tsx	Renders Rive and writes React state into ViewModel properties.
src/components/diabo/DiaboPeek.tsx, DiaboPeekPortal.tsx, HomeDiaboStage.tsx	Additional avatar presentation surfaces.

Technologies: @rive-app/react-canvas, Rive ViewModel instance hooks, typed TypeScript contract.

Data Storage

Primary files:

File	Collection / responsibility
src/lib/mongodb.ts	Lazy cached MongoDB client and database selector.
src/lib/db/chats.ts	chats and messages collections.
src/lib/db/kb.ts	kb_chunks collection with Atlas Vector Search.
src/lib/db/glucose.ts	glucose_logs collection.
src/lib/db/users.ts	users collection.
src/lib/db/reminders.ts	Reminder storage.
src/lib/db/restaurants.ts	Restaurant recommendation cache.
src/lib/db/hotels.ts	Hotel recommendation cache.
src/lib/db/companion.ts	companion_profiles collection.

Technologies: MongoDB Atlas, indexes, TTL-style recommendation cache behavior, Atlas Vector Search.

Health Domain Features

Primary files:

Feature	Files	Implementation
Glucose tracking	src/app/[locale]/glucose/page.tsx, src/components/glucose/GlucoseTracker.tsx, src/app/api/glucose/logs/route.ts, src/lib/db/glucose.ts	Manual glucose entry, units, context labels, chart, metrics, deletion, CSV export.
Weekly glucose summary	src/app/api/glucose/summary/route.ts	Uses generateText () with Diabo safety prompt to summarize 7-day logs.
Restaurant recommendations	src/app/[locale]/restaurants/page.tsx, src/components/restaurants/RestaurantList.tsx, src/app/api/places/restaurants/route.ts, src/lib/restaurant-scorer.ts	Finds places, scores diabetes fit, displays filters, score cards, and map.
Hotel recommendations	src/app/[locale]/hotels/page.tsx, src/components/hotels/HotelList.tsx, src/app/api/places/hotels/route.ts, src/lib/hotel-scorer.ts	Finds lodging, scores travel suitability, displays accessibility and map.
Safety support	src/lib/diabo/persona.ts, src/app/api/chat/route.ts, src/components/chat/ChatPanel.tsx	Emergency prompt override and user-visible safety banner.

Authentication

Primary files:

File	Role
src/lib/auth.ts	Auth.js configuration with Google and credentials providers, JWT sessions, custom login pages.
src/lib/db/users.ts	User creation and lookup, unique email index, password update helper.
src/lib/auth-actions.ts	Server actions for sign-in/sign-out/signup flows.
src/app/api/auth/[...nextauth]/route.ts	Auth.js route handler.
middleware.ts	Protects localized glucose and reminders routes.

Technologies: NextAuth/Auth.js v5, Google OAuth, credentials auth, bcrypt, JWT sessions.

i18n

Primary files:

File	Role
src/i18n/routing.ts	Locale list: fr, ar; French default.
src/i18n/request.ts	Loads locale message JSON asynchronously using Next 16 request API style.
src/i18n/navigation.ts	Localized navigation helpers.
middleware.ts	Locale-aware routing middleware.
src/app/[locale]/layout.tsx	Sets <html lang> and RTL/LTR dir, wraps NextIntlClientProvider.
messages/fr.json, messages/ar.json	UI copy dictionaries.
src/components/LanguageSwitcher.tsx	Locale switcher UI.

Technologies: next-intl, localized App Router segments, RTL-aware layout.

Part 2 - Additional Features Beyond Cahier Des Charges

Feature	What it does	Files	Technologies	Why it was added
Arabic i18n support with RTL	Adds Arabic UI copy and right-to-left layout direction.	messages/ar.json, src/i18n/routing.ts, src/app/[locale]/layout.tsx, src/components/nav/Sidebar.tsx	next-intl, dir="rtl"	The cahier lists multilingual as a bonus; Arabic fits the Maghreb audience.
Dark mode	Lets users switch light/dark/system theme.	src/components/theme/ThemeProvider.tsx, src/components/nav/Sidebar.tsx, src/components/settings/SettingsForm.tsx, src/app/globals.css	next-themes, Tailwind dark classes	Improves comfort for repeated health use.
Google Places API integration	Uses Google Places data for restaurants/hotels when configured, with Overpass fallback.	src/lib/places/google-places.ts, src/lib/overpass.ts, src/lib/overpass-hotels.ts, src/app/api/places/photo/route.ts	Google Places API, Next image/photo proxy, Overpass fallback	Better POI metadata, photos, ratings, and contact fields where a free API key is available.
Push notifications with VAPID	Registers browser push subscriptions for reminders.	src/lib/push.ts, src/lib/notifications/push.ts, src/app/api/push/subscribe/route.ts, src/app/api/notifications/subscribe/route.ts, src/components/reminders/RemindersPanel.tsx, src/components/settings/SettingsForm.tsx, public/sw.js if present at runtime	Web Push, VAPID, Service Worker	Makes reminders actionable without paid SMS/email services.
Restaurant recommendation scoring algorithm	Scores restaurants for diabetes-friendly suitability and carb-load tier.	src/lib/restaurant-scorer.ts, src/lib/db/restaurants.ts, src/lib/restaurants/types.ts, src/components/restaurants/RestaurantList.tsx	AI SDK generateObject, Zod schema, heuristic fallback, Mongo cache	Converts raw POI data into health-domain recommendations.
Hotel recommendation scoring algorithm	Scores hotels for travel suitability and accessibility.	src/lib/hotel-scorer.ts, src/lib/db/hotels.ts, src/components/hotels/HotelList.tsx	AI SDK generateObject, heuristic fallback, Mongo cache	Supports diabetic travel planning beyond chat.
Vector map with user position	Displays recommended places and current/search position on a vector map.	src/components/map/VectorMap.tsx, src/components/restaurants/RestaurantList.tsx, src/components/hotels/HotelList.tsx	MapLibre GL, OpenFreeMap tiles, browser geolocation	Makes recommendations spatial and practical.

Feature	What it does	Files	Technologies	Why it was added
Conversation history sidebar	Lists signed-in user conversations and allows switching.	src/components/chat/ConversationSidebar.tsx, src/app/api/chats/route.ts, src/app/api/chats/[id]/messages/route.ts, src/lib/db/chats.ts	MongoDB, custom browser events	Turns Diabo from a single chat into a persistent companion.
Profile management	Stores diabetes type, goals, body/profile fields, emergency contact, and city.	src/lib/db/companion.ts, src/app/api/companion/profile/route.ts, src/components/onboarding/OnboardingForm.tsx, src/app/api/onboarding/route.ts	MongoDB, Zod validation, local storage bootstrap	Personalizes advice while keeping medical constraints.
Settings page	Centralizes language, theme, notifications, export, and conversation deletion.	src/app/[locale]/settings/page.tsx, src/components/settings/SettingsForm.tsx, src/app/api/settings/export/route.ts, src/app/api/settings/conversations/route.ts	next-intl, next-themes, browser download APIs	Gives users control over data and preferences.
Reminder system	Lets signed-in users create, toggle, and delete reminders.	src/components/reminders/RemindersPanel.tsx, src/app/api/reminders/route.ts, src/app/api/reminders/[id]/route.ts, src/lib/db/reminders.ts	MongoDB, Auth.js session, cron-like time strings	Adds daily adherence support for medication, glucose, hydration, and exercise.
Onboarding flow	Collects diabetes type, main goal, and preferred name before using Diabo.	src/components/onboarding/OnboardingFlow.tsx, src/components/onboarding/OnboardingForm.tsx, src/app/[locale]/onboarding/page.tsx, src/app/api/onboarding/route.ts	React state, localStorage, Mongo sync	Makes the companion feel personal from the first conversation.
Responsive mobile sidebar	Provides collapsible desktop navigation and mobile overlay navigation.	src/components/nav/Sidebar.tsx, src/app/[locale]/layout.tsx	CSS transitions, localStorage, lucide icons	Improves mobile usability for the target mobile-marketing context.
Command palette (Ctrl+K)	Opens a keyboard-searchable route launcher.	src/components/nav/CommandPalette.tsx, src/components/nav/Sidebar.tsx	React keyboard events, localized route labels	Speeds navigation for demos and power users.
Data export/download	Exports glucose CSV and account JSON data.	src/components/glucose/GlucoseTracker.tsx, src/components/settings/SettingsForm.tsx, src/app/api/settings/export/route.ts	Blob downloads, JSON/CSV generation	Supports reportability and user data ownership.
Real-time glucose tracking dashboard	Shows latest glucose, weekly average, target time, out-of-range count, and charts.	src/components/glucose/GlucoseTracker.tsx, src/lib/db/glucose.ts, src/app/api/glucose/logs/route.ts	Recharts, MongoDB, unit conversion	Makes the health domain tangible beyond chat.
Emergency intent detection	Detects urgent phrases and overrides the LLM prompt with emergency escalation.	src/app/api/chat/route.ts, src/lib/diabo/persona.ts	Keyword detection, safety prompt override	Reduces risk in a health assistant.
Safety banners	Shows a visible warning when chat input contains severe-risk terms.	src/components/chat/ChatPanel.tsx	Client-side keyword detection	Reinforces that Diabo is not a medical emergency service.
Skeleton loading states	Uses skeleton placeholders for loading restaurant, hotel, and glucose dashboard content.	src/components/restaurants/RestaurantList.tsx, src/components/hotels/HotelList.tsx, src/components/glucose/GlucoseTracker.tsx, src/app/globals.css	Tailwind utility classes, custom diabo-skeleton style	Keeps the UI stable during network/model delays.

Feature	What it does	Files	Technologies	Why it was added
Route error boundaries	Adds localized error, loading, and not-found boundaries.	src/app/[locale]/error.tsx, src/app/[locale]/loading.tsx, src/app/[locale]/not-found.tsx	Next.js App Router route boundaries	Gives graceful failure states for production demos.
Anonymous-user conversation transfer	Moves a local anonymous chat into a signed-in account.	src/components/chat/ChatPanel.tsx, src/app/api/chats/transfer-anon/route.ts, src/lib/db/chats.ts	localStorage, Mongo ownership update, Auth.js	Avoids losing early conversations after sign-in.
Conversation translation to Arabic	Translates selected chat history into Arabic.	src/components/chat/ConversationSidebar.tsx, src/app/api/chat/translate/route.ts	AI SDK / LLM translation route, browser custom events	Bridges French-first history with Arabic bonus support.
Recommendation cache fallback	Serves cached place recommendations if Overpass fails or times out.	src/app/api/places/restaurants/route.ts, src/app/api/places/hotels/route.ts, src/lib/db/restaurants.ts, src/lib/db/hotels.ts	MongoDB cache, timeout handling	Makes demos resilient to free API instability.
KB citation metadata	Persists RAG citation metadata with assistant messages.	src/lib/diabo/citations.ts, src/app/api/chat/route.ts, src/lib/db/chats.ts	AI SDK message metadata, MongoDB	Improves transparency for RAG-backed answers.

Current Gaps and Naming Differences

Expected by request	Current implementation
src/lib/companion-memory.ts	Memory/profile is implemented through src/lib/db/companion.ts and prompt helpers in src/app/api/chat/route.ts.
src/lib/nlp/	Offline NLP is in notebooks/; live NLP modules are src/lib/emotion.ts, src/lib/embeddings.ts, and src/lib/db/kb.ts.
src/lib/emotion-detection.ts	The implemented file is src/lib/emotion.ts.
PDF cahier with 48 numbered requirements	The PDF is a short unnumbered brief; this document preserves a numbered 48-item extracted matrix for traceability.

Verification Checklist for Future Updates

Before using this documentation in the final report or demo, verify:

- 1 npm run typecheck
- 2 npm run lint
- 3 Chat route still streams with current OpenRouter free model configuration.
- 4 /api/emotion still returns positive/neutral/negative labels with the Hugging Face token configured.
- 5 Restaurants/hotels still work with Google Places or Overpass fallback.
- 6 The generated PDF matches this Markdown after any edits.